AskMyPDF — Extractive Question Answering over PDF Documents

NLP Mini-Project · SVKM's NMIMS, School of Technology Management & Engineering, Hyderabad
Team: G. Vikram (70572300022), B. Dhanush (70022300509) · **Mentor:** Niladri Sath
Repository: github.com/Vikram5002/AskMyPDF

1. Abstract

AskMyPDF answers a plain-language question about any PDF by locating the answer inside the document and reporting it with a confidence score and the page it came from. It uses Hugging Face's `question-answering` pipeline with RoBERTa fine-tuned on SQuAD 2.0. Because the model is **extractive** — it marks where the answer begins and ends rather than generating text — it cannot fabricate a fact, and returns "no answer" when the document does not contain one. It handles long documents by chunking, ~100 languages through XLM-RoBERTa, and scanned books through OCR, as a Streamlit application that runs offline after the first model download.

2. Problem and objectives

Finding one fact in a long document means reading it; keyword search needs the author's exact wording; scanned books hold no searchable text; non-English documents are served worst. Objectives: answer a natural-language question from any PDF, show the confidence and source passage, support long documents, many languages and scanned pages, and decline rather than guess when the answer is absent.

3. Methodology

Stage	Module	What happens
Extraction	`pdf_extractor.py`	`pdfplumber` with `PyPDF2` fallback; word spacing corrected, two-column pages read one column at a time, hyphens and right-to-left scripts repaired
Scan detection	`pdf_extractor.py`	Under 20 words per page means the pages are images → OCR offered
OCR (optional)	`ocr.py`	`pypdfium2` renders pages at 300 dpi; EasyOCR recognises the characters
Chunking	`chunker.py`	Overlapping windows sized to the 512-token limit, each remembering its pages
Inference	`qa_engine.py`	Every chunk is read with the question; the model predicts the answer's start and end

Answer selection. Each chunk also scores "no answer here". Since a confidence score is normalised within its own chunk, chunks are ranked by the **margin** (span score − null score), the standard SQuAD 2.0 approach. If no chunk beats its null option, the system reports no confident answer and lists the closest guesses rather than discarding them.

4. NLP techniques applied

Extractive QA — two distributions over the passage's tokens, for the answer's start and end. **Transfer learning** — pre-trained RoBERTa fine-tuned on SQuAD 2.0; no training was done here. **Sub-word tokenization** — BPE and SentencePiece, measured at ~1.2 tokens/word in English but ~2.7 in Telugu, so chunk size is measured with the model's own tokenizer rather than assumed. **Overlapping segmentation** — chunks share 50 words so a boundary answer stays whole. **Null scoring** for unanswerable questions. **Cross-lingual zero-shot transfer** — XLM-RoBERTa is fine-tuned for QA in English only yet answers in ~100 languages. **Text normalization** — de-hyphenation driven by the document's own vocabulary, Unicode NFKC, bidirectional reordering.

5. Implementation

Python, `transformers` 4.57 (pinned below 5.0, which removed the QA pipeline), PyTorch with CUDA, Streamlit, `pdfplumber`, `PyPDF2`, `sentencepiece` and EasyOCR — seven runtime dependencies. Two selectable models: `deepset/roberta-base-squad2` (124M, English) and `deepset/xlm-roberta-large-squad2` (560M, multilingual). Model metadata is fetched live from the Hugging Face Hub API; inference runs locally or on the hosted Inference API. A GPU is used when present, with CPU fallback on memory exhaustion.

6. Results

Test set	Result
Automated tests (unit, end-to-end, interface)	48 pass
English sample questions	7 / 7
Indian-language questions, 11 languages, clean text	25 / 25
26-language PDF test set	45 / 55
Dense research papers	≈ 4 / 10

Answering takes ~0.2 s per chunk on CPU and 0.32 s on GPU for the larger model; OCR costs ~20 s per page.

Testing on real documents, rather than samples we generated ourselves, produced every important correction. Research papers exposed four extraction faults: word spacing that glued lines into `WeusedtheAdamoptimizer` (2,017 → 6,183 words recovered), two-column pages read straight across, hyphen removal turning `self-attention` into `selfattention`, and chunks of 900–2,400 tokens against a 512 limit. The clearest single result: "*What does BERT stand for?*" answered "*feature-based approach*" at 96% confidence, because the abstract and introduction were spliced together line by line; after the column fix it answers "*Bidirectional Encoder Representations from Transformers*" at 98%.

A 39-page scanned Telugu book yielded 10 words of text — a watermark — and drove the OCR path; six pages gave 2,681 usable words and correct answers. The 26-language set showed right-to-left scripts extracting backwards (repaired: Arabic, Urdu and Sindhi went 0/6 → 6/6) and that Indic conjunct letters are lost by the PDF format itself, which no code can recover from the text layer — though reading the page as an image can, taking Telugu, Kannada and Bengali from 1/2 to 2/2.

7. Limitations

Extractive only: no summarising, counting or calculation. It reads one chunk at a time, so facts spread across distant pages cannot be combined. Tables and complex layouts arrive as flat rows of digits, which is why research-paper accuracy is about 4 in 10. Answers stay in the document's language. EasyOCR has no model for Gujarati, Malayalam, Odia or Punjabi. The multilingual model needs ~2.5 GB of free memory.

8. Conclusion

A working question-answering system over PDFs was built and measured: it finds the answer, states its confidence, and shows the page — across ~100 languages and on scanned books. Three findings stand out. Clean test data hides bugs: every fault that mattered was invisible until real documents were used. A confidently wrong answer is usually a text-extraction problem, not a model problem. And confidence must be earned — comparing chunks requires the margin over saying nothing, not the raw score.

Future work: translate answers into the question's language; batch chunks on the GPU; add Tesseract OCR for the scripts EasyOCR omits; table-aware extraction for numeric questions.

References

1. Rajpurkar et al., *Know What You Don't Know: Unanswerable Questions for SQuAD*, ACL 2018.
2. Liu et al., *RoBERTa: A Robustly Optimized BERT Pretraining Approach*, 2019.
3. Conneau et al., *Unsupervised Cross-lingual Representation Learning at Scale*, ACL 2020.
4. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.
5. Hugging Face: `deepset/roberta-base-squad2`, `deepset/xlm-roberta-large-squad2`.